

LOOK BEFORE YOU FLOW: INFERENCE-TIME RISK STEERING FOR FROZEN FLOW-MATCHING VLAs

Anonymous authors

Paper under double-blind review

ABSTRACT

We introduce **MARS** (Multi-risk Action Repulsion and Steering), a training-free inference-time framework that improves the safety of frozen flow-matching vision-language-action (VLA) policies by translating semantic risk knowledge from a vision-language model into bounded action-space corrections. Deploying VLA policies in physical environments demands avoiding collisions with fragile objects, bystanders, and other scene hazards, yet existing safety methods either require retraining the policy, depend on known object geometry and dynamics models, or only specify constraints without modifying the continuous action output. MARS addresses this gap through two components: *Grounded Risk Verbalization* (GRV), which uses a frozen VLM to identify scene risks and express each as a counterfactual action prompt in the VLA’s native instruction format, and *Trajectory-Anchored Flow Repulsion* (TAFR), which evaluates the frozen VLA under both nominal and counterfactual instructions at a shared anchor state and accumulates per-risk velocity residuals in a correction state decoupled from the model’s evaluation trajectory, preventing guidance perturbations from compounding across ODE integration steps. We evaluate MARS on two simulation benchmarks—VLA-Arena and LIBERO-Safety—comparing against four VLA baselines spanning 0.45B to 3B parameters, as well as on real-world manipulation tasks with two robot platforms. On LIBERO-Safety, MARS improves safety over the unguided $\pi_{0.5}$ policy by approximately 10% while preserving task success, without modifying any VLM or VLA parameters.

1 INTRODUCTION

Recent advances in robot learning have accelerated the pursuit of generalist robots that can operate across tasks and environments. Vision-language-action (VLA) policies provide a common interface for this goal by mapping visual observations and natural-language instructions to robot actions. Generalist VLAs increasingly combine pretrained visual and linguistic representations with generative decoders that produce continuous actions. In particular, $\pi_{0.5}$ combines high-level semantic prediction with a low-level controller based on flow matching to generate continuous action sequences, while GR00T N1 couples a VLM reasoning module with a flow-matching Diffusion Transformer to produce continuous action chunks across robot embodiments (Physical Intelligence et al., 2025; NVIDIA et al., 2025). Flow matching learns a vector field whose ordinary differential equation (ODE) transports noise to a data sample (Lipman et al., 2023), enabling policies to generate continuous action chunks at high control frequencies without discretizing each control dimension.

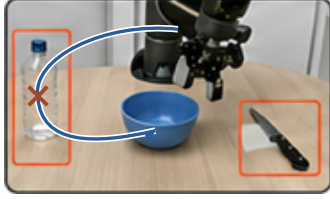
Task success alone, however, does not suffice for deployment in physical environments. A robot may complete an instruction while colliding with nearby geometry, disturbing an object that should remain untouched, or endangering nearby agents. We define *safety* as the ability to execute the requested task while avoiding physical violations involving the robot, protected objects, and the surrounding scene. Recent benchmarks have begun evaluating such violations separately from nominal task success (Cui et al., 2026). In practice, risk objects change constantly—a wine bottle may appear on a countertop one session but not the next—so a safety mechanism that requires retraining the VLA for each new risk object is impractical. What is needed is a general, training-free interface that can be applied to any frozen flow-matching VLA.

Task completion alone does not ensure safe interaction

Task: “pick up the bowl”

Same scene; schematic paths

(a) Task-only VLA

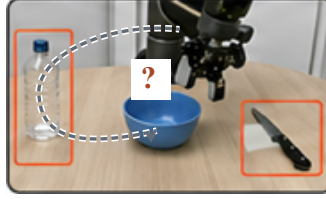


Possible contact with bottle

“pick up the bowl”

Task pursuit can disturb non-target objects.

(b) Avoidance prompting

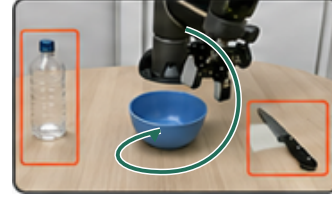


Language constraint only

“pick up the bowl and avoid the bottle and the knife”

Avoidance is not assured.

(c) MARS (ours)



Risk-aware action correction

“pick up the bottle”
“pick up the knife”

References only; steer actions without retraining

MARS: scene risks → counterfactual references → action correction; no model updates

Figure 1: Motivation for MARS. In a shared manipulation scene, the robot must pick up the bowl while avoiding unwanted interactions with nearby objects. (a) A task-only policy may pursue the goal while disturbing a non-target object. (b) Appending avoidance clauses specifies a language constraint but does not ensure the corresponding action correction. (c) MARS represents scene risks as positive-form counterfactual prompts, uses them only as references, and steers a frozen VLA at inference time. Overlaid paths illustrate the conceptual comparison; they are not measured rollouts.

Consider the example in Figure 1: a robot is instructed to “pick up the bowl” while a bottle sits near the gripper and the target. The flow-matching VLA generates the entire action chunk as a single continuous sequence; without explicit safety constraints, the planned trajectory may sweep through the bottle, knocking it over. The task succeeds, but the environment is damaged. A natural first attempt is prompt injection—appending an avoidance clause such as “pick up the bowl and avoid the bottle” to the task instruction. However, compound avoidance instructions are out of distribution for VLAs trained on simple action commands, and this modification degrades rather than improves task performance. The problem worsens with multiple simultaneous risks: if both a bottle and a knife are present, the instruction becomes “pick up the bowl and avoid the bottle and avoid the knife,” further straining the policy’s instruction-following capability. These failure modes motivate an alternative interface that connects semantic risk knowledge to continuous action correction without modifying the task instruction or the policy parameters.

Existing approaches to VLA safety each address part of this problem but leave a gap at their intersection. SafeVLA formulates safety as a constrained Markov decision process and applies safe reinforcement learning, but its implementation fine-tunes a causal transformer over a discrete action space (Zhang et al., 2025a); extending this pipeline to the continuous action chunks produced by flow-matching VLAs would require redesigning cost modeling, critic learning, and policy optimization. VLSA introduces AEGIS, a safety layer that converts visual and depth observations into geometric obstacle models and filters nominal actions through control barrier functions (Hu et al., 2025). AEGIS provides geometric constraints but depends on known object geometry, robot dynamics models, and online optimization, and cannot express risks that are semantic rather than purely spatial—the wine bottle is not merely an obstacle to circumnavigate but a fragile object that should not be disturbed. VLM-based methods can identify relevant scene entities and represent safety constraints as natural-language specifications (Santos et al., 2024; Feng et al., 2025), but they operate at the level of constraint description: they do not define a mechanism for modifying the continuous action output of a flow-matching policy, nor do they bound the magnitude of any intervention. No existing method bridges semantic risk recognition to bounded action-space correction for frozen flow-matching VLAs without additional training.

We therefore ask: *How can semantic risk knowledge from a VLM be translated into bounded action-space corrections for a frozen flow-matching VLA without additional training?*

We propose **Multi-risk Action Repulsion and Steering (MARS)**, an inference-time framework that connects VLM risk recognition to the continuous action space of a frozen flow-matching VLA. MARS first applies **Grounded Risk Verbalization (GRV)** to represent each visually grounded risk as a structured record containing a counterfactual behavior prompt—a simple action command that expresses the undesired interaction (e.g., “pick up the bottle” rather than “avoid the bottle”). Given shared initial noise, the frozen VLA generates a nominal action chunk from the task instruction and counterfactual action chunks from the risk prompts. **Trajectory-Anchored Flow Repulsion (TAFR)** then guides the flow-matching ODE integration: at each step, clipped velocity residuals between the nominal and counterfactual prompts accumulate in a correction state that is decoupled from the model’s evaluation trajectory, preventing guidance perturbations from compounding across integration steps. Because both the VLM and VLA remain frozen, MARS is compatible with any flow-matching VLA and requires no additional training data, reward modeling, or policy optimization.

On the LIBERO-Safety benchmark, MARS improves safety over $\pi_{0.5}$ by approximately 10% while preserving task success. Our contributions are as follows:

- We propose **MARS**, a training-free inference-time framework that improves the safety of frozen flow-matching VLA policies without updating VLM or VLA parameters. MARS requires no additional training and is compatible with any flow-matching-based VLA.
- We introduce **GRV**, which uses a VLM to identify scene risks and produce structured counterfactual action prompts, and **TAFR**, which guides the flow-matching ODE via trajectory-anchored velocity residuals to produce bounded action corrections. Together, these modules translate semantic risk knowledge into explicit, bounded action-space interventions.
- We evaluate MARS on two complementary benchmarks—**VLA-Arena** and **LIBERO-Safety**—comparing against four VLA baselines and validating on two real-world robot platforms. MARS improves safety metrics over $\pi_{0.5}$ by approximately 10% on LIBERO-Safety while maintaining task performance.

2 RELATED WORK

Vision-language-action policies. Embodied learning has progressed from task-specific visuomotor policies toward foundation-scale models that connect visual observations, language instructions, and robot actions. RT-2 and OpenVLA adapt pretrained vision-language models by tokenizing actions and training autoregressively (Brohan et al., 2023; Kim et al., 2024). Diffusion Policy models multimodal action distributions through iterative denoising and executes continuous action chunks in a receding-horizon loop (Chi et al., 2023). Flow matching learns a time-dependent vector field whose ordinary differential equation transports noise to a data sample (Lipman et al., 2023). π_0 attaches a flow-matching action expert to a pretrained VLM (Black et al., 2024), $\pi_{0.5}$ extends this with open-world generalization (Physical Intelligence et al., 2025), GR00T N1 couples a VLM reasoning module with a flow-matching Diffusion Transformer (NVIDIA et al., 2025), and X-VLA adopts embodiment-specific soft prompts (Zheng et al., 2025). The flow-matching formulation produces continuous action chunks over multiple integration steps, creating several possible points at which an external signal can intervene.

Inference-time guidance for flow-matching policies. Recent methods intervene at different stages of the flow-matching generation process. During sampling, Counterfactual Action Guidance mixes conditioned and language-unconditioned VLA outputs to reduce visual shortcuts (Fang et al., 2026b), and TAG contrasts target-present and target-erased visual conditions to improve spatial grounding (Zhou et al., 2026). Learned critics provide gradient-based steering: VQActFlow uses language guidance and a learned critic to steer discrete action modes (Zhao et al., 2026), while Guided Action Flow applies Q-function gradients with clipping and uncertainty gating to a frozen flow-matching VLA (Yang et al., 2026). POTR constrains intermediate flow guidance with an orthogonal trust region to improve action-chunk continuity (Fang et al., 2026a). OmniGuide defines energy functions over Cartesian trajectories to shape the generative flow during inference (Song et al., 2026). Flow Reversal Steering reverses the flow integration to map coarse VLM-generated reference actions into nearby in-distribution VLA action modes (Tang et al., 2026). These methods modify the velocity field, constrain intermediate estimates, reshape the noise distribution, or invert

the flow. MARS differs in mechanism: at each ODE step it evaluates the velocity field under both nominal and counterfactual instructions at a shared anchor state, accumulating per-risk velocity residuals in a correction state decoupled from the model’s evaluation trajectory. This trajectory-anchored guidance requires no critic, no gradient computation, and no geometric model.

Safety for embodied agents. Training-time approaches embed safety objectives into the policy-learning process. SafeVLA formulates VLA alignment as a constrained Markov decision process and applies safe reinforcement learning over elicited unsafe behaviors (Zhang et al., 2025a). Its implementation fine-tunes a causal transformer over a discrete action space; extending this pipeline to the continuous action chunks of flow-matching VLAs would require redesigning cost modeling, critic learning, and policy optimization.

Inference-time methods preserve the frozen VLA while modifying or filtering its actions. AEGIS converts depth observations into geometric obstacle models and applies control-barrier-function quadratic programs for post-hoc action filtering (Hu et al., 2025). Neuro-symbolic safety guidance embeds trajectory-level barrier constraints within flow-matching generation (English et al., 2026), and Barrier Enhanced Flow Matching similarly constrains action chunks during generation with a distributional bound (Sinaei et al., 2026). Any-Body Guard operates in configuration space (Beaudin et al., 2026), and ProbeAct combines internal object probes with hierarchical barrier filtering for run-time recovery (Zhang et al., 2026). These methods provide spatial safety enforcement but require known object geometry, robot dynamics models, or learned monitors.

A parallel line uses VLMs to specify safety constraints in natural language. VLMs can update reachability-based safety representations from language feedback (Santos et al., 2024), and language-conditioned filtering grounds free-form constraints in object-level 3D maps (Feng et al., 2025). These approaches operate at the level of constraint specification: they identify what should be avoided but do not define a mechanism for modifying the continuous output of a flow-matching policy.

MARS bridges semantic risk recognition and action-space correction. It uses a VLM to identify risks and express them as counterfactual action prompts, then translates these prompts into bounded end-point corrections via shared-noise flow comparison—without additional training, geometric models, or learned critics.

3 MARS: MULTI-RISK ACTION REPULSION AND STEERING

We propose MARS (Figure 2), a *training-free* inference-time framework that uses a vision-language model (VLM) to mitigate risk in flow-matching robot policies. Formally, let $o_t = [I_t^1, \dots, I_t^n]$ be the sequence of multi-view observations at replanning step t , and let c_t^+ be the natural-language task instruction. A frozen flow-matching VLA F_θ maps a shared initial noise ξ to an action chunk $P = (P_h)_{h=1}^H$, where each $P_h \in \mathbb{R}^d$ encodes a single-step action and H is the chunk horizon. The mapping from noise to action is deterministic for a given condition: $P = F_\theta(o_t, c_t^+; \xi)$. The goal of MARS is to produce a corrected action chunk $\{P_h^{\text{corr}}\}_{h=1}^H$ that follows c_t^+ without modifying the VLA parameters θ . Sampling from F_θ integrates a conditional velocity field $v_\theta(\mathbf{x}, \tau; o_t, c)$, where the flow state $\mathbf{x} \in \mathbb{R}^{H \times d}$ represents an action chunk of horizon H in the backbone’s normalized continuous action coordinates; MARS intervenes on this field rather than on θ . We use flow time $\tau = 1$ for noise and $\tau = 0$ for actions, with $K \geq 1$ Euler steps, $\tau_k = 1 - k/K$, and $\Delta t = -1/K$.

To achieve this, MARS comprises two components: **Grounded Risk Verbalization (GRV)** (§3.1), which converts visually grounded risks into simple, VLA-compatible counterfactual action prompts, and **Trajectory-Anchored Flow Repulsion (TAFR)** (§3.2), which guides the flow-matching ODE integration via trajectory-anchored velocity residuals to produce bounded action corrections. Both the VLM and VLA parameters remain frozen throughout; MARS requires no additional training, reward modeling, or policy optimization.

3.1 GROUNDED RISK VERBALIZATION

GRV introduces a VLM to enhance the action safety of a frozen VLA without requiring policy updates or additional training. A direct alternative is prompt injection: the VLM analyzes the current

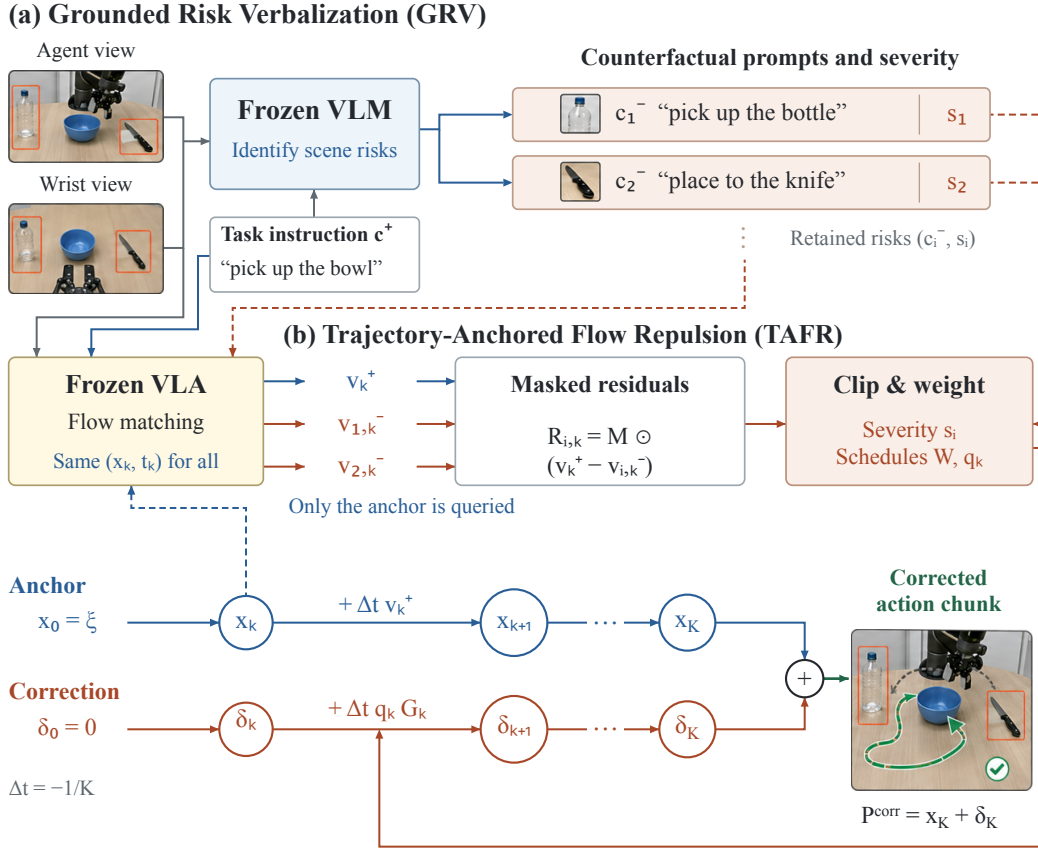


Figure 2: Overview of MARS. Given multi-view observations o_t and a task instruction c_t^+ , **GRV** (§3.1) uses a frozen VLM to identify scene risks and produce counterfactual action prompts. The frozen VLA evaluates both the nominal and counterfactual prompts from shared noise ξ . **TAFR** (§3.2) accumulates trajectory-anchored velocity residuals along the ODE integration to produce a bounded corrected action chunk. Both the VLM and VLA remain frozen throughout; no training or parameter update is required.

scene, identifies potential risks, and augments the nominal task instruction with an explicit avoidance clause. For example, given the instruction “pick up the bowl to the plate,” the VLM may produce “pick up the bowl and avoid the wine bottle.” This compound prompt requires the VLA to complete the original task while grounding an additional object and satisfying an avoidance constraint. However, in our benchmark, the increased instruction complexity reduces task success by approximately 5% without reliably inducing avoidance behavior. Rather than requiring the VLA to execute a compound task-and-avoidance instruction, GRV assigns scene and risk interpretation to the VLM and represents each identified risk with a simple, VLA-compatible counterfactual action prompt for downstream steering. GRV operates as two coupled components: a *VLM interface* that governs prompt construction and risk-record generation, and an *invocation scheduler* that determines when the VLM is called based on the current risk state.

VLM interface. GRV instantiates a frozen VLM $\mathcal{V}_\phi$ (Qwen3-VL-2B-Instruct (Qwen Team, 2025)) that receives agent-view and wrist-view images at 256×256 resolution and produces structured risk records via JSON schema enforcement. Given o_t and the task instruction c_t^+ , the VLM identifies candidate risk objects—non-target entities whose disturbance would violate the task specification—and returns, for each candidate i , a record

$$\mathcal{V}_\phi(o_t, c_t^+) \ni (e_{t,i}, \beta_{t,i}, c_{t,i}^-, g_{t,i}, p_{t,i}, \rho_{t,i}), \quad (1)$$

where $e_{t,i}$ names the entity, $\beta_{t,i}$ is its image region, $c_{t,i}^-$ is a counterfactual action prompt, and $(g_{t,i}, p_{t,i}, \rho_{t,i}) \in [0, 1]^3$ scores grounding confidence, proximity, and semantic relevance. A candidate is retained when $g_{t,i} \geq \tau_g$ and $\rho_{t,i} \geq \tau_\rho$, and carries severity $s_{t,i}$:

$$\mathcal{I}_t = \{i : g_{t,i} \geq \tau_g, \rho_{t,i} \geq \tau_\rho\}, \quad s_{t,i} = g_{t,i} p_{t,i} \rho_{t,i}. \quad (2)$$

When more than $L_{\max}$ candidates pass both thresholds, GRV keeps those with the largest severity, so the retained count $L = |\mathcal{I}_t|$ never exceeds $L_{\max}$. Each counterfactual prompt $c_{t,i}^-$ expresses the undesired interaction as a positive action command (e.g., “pick up the bottle” rather than “avoid the bottle”), giving the VLA a familiar instruction format. TAFR does not execute these prompts; it uses them only as reference directions for suppressing the corresponding behavior. The retained records $\{(c_{t,i}^-, s_{t,i})\}_{i \in \mathcal{I}_t}$ and their severity weights constitute the interface to TAFR. When $\mathcal{I}_t = \emptyset$, MARS returns the unmodified VLA output. The complete prompt template and parsing details are provided in Appendix A.

Invocation scheduler. The VLM is not invoked at every replanning step. Under normal conditions, GRV calls the VLM once every N replanning cycles (default $N=4$) and reuses the most recent risk records for the intervening cycles. To hide VLM latency, GRV pipelines the VLM forward pass with action execution: while the robot executes the current corrected chunk over H_{exec} steps, the VLM begins risk assessment on the latest observation asynchronously. By the time the chunk completes, the updated risk records are already available, allowing TAFR to compute the next correction without stalling on VLM inference. When GRV detects potentially risky moving objects, it reduces the invocation interval to $N=1$, querying the VLM at every replanning cycle. This trades additional VLM inference cost for reduced response latency in dynamic scenes. Appendix B defines the request, motion, and failure rules.

3.2 TRAJECTORY-ANCHORED FLOW REPULSION

Velocity residuals. The central mechanism of TAFR is that a flow-matching VLA, evaluated at the same ODE state under different language conditions, produces velocity fields whose difference encodes the instantaneous behavioral divergence induced by the instruction change. At each ODE step k , TAFR evaluates the frozen VLA at the current anchor state $\mathbf{x}_k$ and flow time τ_k under both the nominal instruction c_t^+ and each retained counterfactual instruction $c_{t,i}^-$:

$$\mathbf{v}_k^+ = v_\theta(\mathbf{x}_k, \tau_k; o_t, c_t^+), \quad \mathbf{v}_{i,k}^- = v_\theta(\mathbf{x}_k, \tau_k; o_t, c_{t,i}^-), \quad \mathbf{R}_{i,k} = \mathbf{M} \odot (\mathbf{v}_k^+ - \mathbf{v}_{i,k}^-). \quad (3)$$

The binary mask $\mathbf{M} \in \{0, 1\}^{H \times d}$ restricts the correction to actionable dimensions (e.g., Cartesian position and orientation) while excluding dimensions such as discrete gripper state. All $L+1$ conditions share the same observation, so they differ only in their language embedding and can be evaluated as a single batch. Each residual encodes the direction in which the nominal behavior diverges from the counterfactual at step k . When multiple risks are present, their residuals carry distinct avoidance directions; TAFR processes each independently before combining. A raw residual can be arbitrarily large where the two conditions disagree sharply, so TAFR clips each component to a finite cap $b_a \geq 0$ in normalized velocity units:

$$R_{i,k,h,a}^{\text{clip}} = \max\{-b_a, \min\{b_a, R_{i,k,h,a}\}\}. \quad (4)$$

The caps are per-dimension, so translation and rotation can be limited independently. Together with the finite guidance weights introduced below, they cap how far the correction can move any action coordinate, whatever the VLM reports.

Guidance scheduling. TAFR modulates the correction along two axes. Along the action-chunk horizon, a power-ramp weight matrix $\mathbf{W} \in \mathbb{R}^{H \times d}$ concentrates guidance on later steps where trajectory divergence is largest:

$$W_{h,a} = w_a \cdot \left(\frac{h}{H}\right)^{\gamma_{\text{eff}}}, \quad \gamma_{\text{eff}} = \gamma(1-r), \quad r = 1 - \prod_{i \in \mathcal{I}_t} (1-s_i), \quad (5)$$

where w_a is the per-dimension maximum guidance weight, γ is the base exponent, $s_i \equiv s_{t,i}$ is the per-risk severity from Eq. equation 2 (records stay fixed within a replanning cycle, so we drop t),

and r is the noisy-OR aggregate risk. As risk increases, γ_{eff} decreases toward zero, flattening the ramp so that even the first action step receives full guidance. Along the ODE integration axis, a temporal weight

$$q_k = \left(\frac{k+1}{K} \right)^\alpha \quad (6)$$

ramps guidance strength over the K Euler steps, concentrating the correction on later integration steps where the velocity field carries more directional signal. The two schedules compose multiplicatively, jointly determining the guidance magnitude at each combination of integration step and chunk position.

Trajectory-anchored integration. A naive approach would apply guidance corrections directly to the ODE state, causing each perturbation to shift subsequent velocity evaluations and amplify errors recursively. TAFR instead maintains two decoupled states: an *anchor* $\mathbf{x}_k$ that follows the unperturbed positive-prompt trajectory, and a *correction* δ_k that accumulates the safety adjustment independently. Given shared initial noise ξ and K Euler steps with step size $\Delta t = -1/K$:

$$\mathbf{x}_0 = \xi, \quad \delta_0 = \mathbf{0}. \quad (7)$$

At each step $k = 0, \dots, K-1$, the weighted guidance signal is

$$\mathbf{G}_k = \mathbf{W} \odot \sum_{i \in \mathcal{I}_t} s_i \mathbf{R}_{i,k}^{\text{clip}}, \quad (8)$$

and the two states update independently:

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \Delta t \mathbf{v}_k^+, \quad (9)$$

$$\delta_{k+1} = \delta_k + \Delta t q_k \mathbf{G}_k. \quad (10)$$

Crucially, all velocities $\mathbf{v}_k^+$ and $\mathbf{v}_{i,k}^-$ are evaluated at $\mathbf{x}_k$, not at $\mathbf{x}_k + \delta_k$: the accumulated correction never feeds back into velocity evaluations, eliminating recursive error amplification. When $\mathcal{I}_t = \emptyset$, all severities are zero, $\delta_K = \mathbf{0}$, and the output reduces exactly to the unguided trajectory. The corrected chunk $P^{\text{corr}} = \mathbf{x}_K + \delta_K$ is decoded with the backbone’s usual action postprocessor, and its first $1 \leq H_{\text{exec}} \leq H$ actions are executed before replanning. A variant that instead evaluates velocities at the corrected state $\mathbf{x}_k + \delta_k$ reintroduces the feedback that anchoring removes; Appendix D gives its form and why MARS excludes it.

4 EXPERIMENTS

We evaluate MARS on complementary simulation and real-world benchmarks that test task completion and safety constraint satisfaction, comparing against four VLA baselines that span different architectural designs.

Benchmarks. We conduct experiments on two simulation benchmarks and a real-world task suite:

(1) VLA-Arena (Zhang et al., 2025b) provides a systematic evaluation framework covering 170 tasks across 11 specialized suites organized into four evaluation dimensions: *Safety*, *Distractors*, *Extrapolation*, and *Long-horizon reasoning*. Each suite features three difficulty levels (L0–L2), with fine-tuning restricted to L0 to test out-of-distribution generalization. We focus exclusively on the **Safety** dimension (five suites: StaticObstacles, DynamicObstacles, HazardAvoidance, Cautious-Grasp, StatePreservation), as it directly measures the constraint-satisfaction behavior that MARS targets; the remaining three dimensions evaluate orthogonal capabilities (distractor robustness, compositional generalization, long-horizon planning) that are outside the scope of our safety-oriented intervention. We report *Success Rate* (SR, %) and *Cumulative Cost* (CC) in the benchmark’s native units. The benchmark builds on the LIBERO simulation infrastructure.

(2) LIBERO-Safety (Cui et al., 2026) is a comprehensive benchmark for evaluating physical and semantic safety in VLA models. It extends the LIBERO benchmark (Liu et al., 2024) with safety-critical scenarios organized into five task suites: Affordance-Aware Grasping (AAG), Human–Robot Interaction (HRI), Obstacle Avoidance, Physical Safety, and Semantic Safety Reasoning. Each suite contains five tasks at three difficulty levels: L0 (nominal workspace), L1 (spatial perturbations such

as shifted object positions or human-proxy intrusions), and L2 (held-out configurations unseen during training), yielding 75 unique task configurations. The benchmark provides 19,664 collision-free demonstrations generated via a keypose-driven pipeline with domain randomization. We report four metrics: *Success Rate* (SR, the primary metric—requires goal completion with zero constraint violations), *Collision Rate* (CR), *Execution Time*, and *Log-Dimensionless Jerk* (LDLJ, trajectory smoothness). All experiments use the Franka Panda arm in the robosuite/MuJoCo simulator.

(3) Real-world safety tasks comprise five tabletop task families on the Dobot CR-10 and Galaxea A1X: affordance-aware grasping, obstacle avoidance, hazard avoidance, cautious grasp, and state preservation. We select short manipulation tasks from the corresponding LIBERO-Safety and VLA-Arena suites and adapt their props for physical execution. We report task completion, safe completion, violation, and abort rates; task details are given in Section 4.2.

Baselines. We compare MARS against four VLA models:

(1) $\pi_{0.5}$ (Physical Intelligence et al., 2025) is a 3B-parameter VLA from Physical Intelligence. It uses a PaLI-based VLM backbone and a low-level controller based on flow matching to generate continuous action sequences. $\pi_{0.5}$ is pre-trained on the largest proprietary robot dataset reported to date (903M+ timesteps across 10+ embodiments) and supports chain-of-thought reasoning before acting.

(2) StarVLA-PI (Zheng et al., 2026) is a VLA variant built on the StarVLA modular codebase. It pairs a Qwen3-VL vision-language backbone with a π_0 -style flow-matching action expert. The StarVLA framework follows a plug-and-play design that decouples the VLM backbone, action head, data pipeline, and evaluation into interchangeable modules, enabling controlled comparisons across architectures under identical training and evaluation protocols.

(3) StarVLA-GR00T (Zheng et al., 2026) adopts a dual-system architecture within the same StarVLA codebase: the VLM backbone serves as System 2 for slow visual-language reasoning, while a DiT-based flow-matching module serves as System 1 for continuous action generation, following the GR00T N1.5 design (NVIDIA et al., 2025). This variant enables direct comparison of the flow-matching dual-system paradigm against the autoregressive approach of StarVLA-PI under matched training conditions.

(4) SmolVLA (Allard et al., 2025) is a 450M-parameter compact VLA from the LeRobot project (Hugging Face). It uses a trimmed SmolVLM-2 backbone with a Transformer-based flow-matching action expert and asynchronous inference that decouples perception from action generation. SmolVLA is trained entirely on community-contributed open datasets, making it the smallest and most accessible baseline in our comparison.

For all baselines, we use the officially released checkpoints and follow each model’s recommended fine-tuning protocol on the benchmark training splits. MARS is applied as a plug-in wrapper over each backbone, requiring no modification to the baseline model weights or architecture.

4.1 SIMULATION RESULTS

VLA-Arena. Table 2 presents per-suite results on VLA-Arena’s Safety dimension, comparing MARS (applied as a plug-in wrapper over $\pi_{0.5}$) against four VLA baselines spanning different architectures and scales. Among the baselines, StarVLA-GR00T achieves the highest L0 success rates on most suites (80% StaticObstacles, 60% DynamicObstacles) but degrades sharply under distribution shift—dropping to 18% and 4.5% on StaticObstacles at L1 and L2. All baselines incur substantial cumulative costs at higher difficulty levels; StarVLA-PI and StarVLA-GR00T accumulate $CC > 98$ on CautiousGrasp L1, indicating frequent unsafe contact despite moderate task completion. SmolVLA exhibits the lowest CC in several suites but also the lowest SR, suggesting that its limited capability avoids cost primarily by failing to attempt tasks. MARS reduces cumulative cost most dramatically on HazardAvoidance: CC drops from 7.61 to 0.10 at L0 (98.7% reduction), from 13.88 to 2.36 at L1, and from 16.60 to 2.66 at L2, while improving success rate—notably gaining +22 percentage points at L2 (28%→50%). On StaticObstacles, MARS eliminates all cumulative cost at L1 (30.42→0.00) and improves SR from 64% to 78%, outperforming all baselines at this difficulty level on both metrics. At L0, MARS improves SR from 84% to 94% with zero cumulative cost. CautiousGrasp shows a mixed picture: MARS improves L0 SR by 12 points (38%→50%) but un-

Table 1: Results on LIBERO-Safety. We report Success Rate (SR $\uparrow$) and Collision Rate (CR $\downarrow$) averaged across 5 tasks per suite. L0/L1/L2 denote increasing difficulty levels. † MARS applied as a plug-in wrapper over the corresponding VLA backbone.

Method	AAG		HRI		Obstacle		Physical		Semantic	
	SR	CR	SR	CR	SR	CR	SR	CR	SR	CR
<i>Difficulty Level L0 (Nominal)</i>										
$\pi_{0.5}$	94	–	72	–	38	–	44	–	–	–
StarVLA-PI	–	–	–	–	–	–	–	–	–	–
StarVLA-GR00T	–	–	–	–	–	–	–	–	–	–
SmolVLA	–	–	–	–	–	–	–	–	–	–
$\pi_{0.5} + \text{MARS}^\dagger$	100	100	80	80	42	42	46	46	–	–
StarVLA-PI + $\text{MARS}^\dagger$	–	–	–	–	–	–	–	–	–	–
StarVLA-GR00T + $\text{MARS}^\dagger$	–	–	–	–	–	–	–	–	–	–
SmolVLA + $\text{MARS}^\dagger$	–	–	–	–	–	–	–	–	–	–
<i>Difficulty Level L1 (Perturbations)</i>										
$\pi_{0.5}$	80	–	86	–	40	–	40	–	–	–
StarVLA-PI	–	–	–	–	–	–	–	–	–	–
StarVLA-GR00T	–	–	–	–	–	–	–	–	–	–
SmolVLA	–	–	–	–	–	–	–	–	–	–
$\pi_{0.5} + \text{MARS}^\dagger$	78	78	88	88	42	42	40	40	–	–
StarVLA-PI + $\text{MARS}^\dagger$	–	–	–	–	–	–	–	–	–	–
StarVLA-GR00T + $\text{MARS}^\dagger$	–	–	–	–	–	–	–	–	–	–
SmolVLA + $\text{MARS}^\dagger$	–	–	–	–	–	–	–	–	–	–
<i>Difficulty Level L2 (Held-out)</i>										
$\pi_{0.5}$	48	–	84	–	38	–	36	–	36	–
StarVLA-PI	–	–	–	–	–	–	–	–	–	–
StarVLA-GR00T	–	–	–	–	–	–	–	–	–	–
SmolVLA	–	–	–	–	–	–	–	–	–	–
$\pi_{0.5} + \text{MARS}^\dagger$	50	50	86	86	42	42	36	36	–	–
StarVLA-PI + $\text{MARS}^\dagger$	–	–	–	–	–	–	–	–	–	–
StarVLA-GR00T + $\text{MARS}^\dagger$	–	–	–	–	–	–	–	–	–	–
SmolVLA + $\text{MARS}^\dagger$	–	–	–	–	–	–	–	–	–	–

derperforms at L1, suggesting that the tight manipulation tolerances in this suite can cause MARS’s safety corrections to interfere with precise grasping. DynamicObstacles reveals a limitation of the current GRV pipeline: while MARS improves L0 SR (54% $\rightarrow$ 64%), it incurs higher cumulative cost than $\pi_{0.5}$ across all levels. We attribute this to the VLM’s difficulty in accurately localizing rapidly moving obstacles, which can produce misaligned avoidance directions (see §4.3). On StatePreservation, MARS improves SR from 26% to 44% at L1 and from 28% to 34% at L2 while maintaining comparable cumulative cost.

4.2 REAL-WORLD EXPERIMENTS

To validate that MARS transfers beyond simulation, we evaluate on two real-world robot platforms with complementary form factors:

(1) Dobot CR-10 is a 6-DoF industrial collaborative robot arm with a 10kg payload capacity and 1525 mm reach. Its large workspace enables tabletop manipulation tasks with multiple objects spread across a wide area, testing MARS’s ability to handle spatially distributed risks.

Table 2: Per-suite results on VLA-Arena (**Safety** dimension). We report Success Rate (SR $\uparrow$, %) and Cumulative Cost (CC $\downarrow$) at three difficulty levels: L0 (nominal), L1 (spatial perturbations), and L2 (held-out configurations). Best results per suite and level are **bolded**. $\dagger$ MARS applied as a plug-in wrapper over $\pi_{0.5}$.

Suite	Method	L0 (Nominal)		L1 (Perturbed)		L2 (Held-out)	
		SR	CC	SR	CC	SR	CC
StaticObstacles	$\pi_{0.5}$	84	0.00	64	30.42	50	54.94
	StarVLA-PI	60	0.00	24	14.30	0	21.38
	StarVLA-GR00T	80	0.00	18	9.06	4.5	29.58
	SmolVLA	14	0.00	0	8.80	0	2.60
	$\pi_{0.5} + \text{MARS}^\dagger$	94	0.00	78	0.00	54	80.82
	StarVLA-PI + MARS †	70	0.00	30	0.00	20	0.00
	StarVLA-GR00T + MARS †	76	0.00	28	0.00	20	0.00
	SmolVLA + MARS †	46	0.00	10	5.10	0	0.00
DynamicObstacles	$\pi_{0.5}$	54	4.12	42	10.94	24	3.32
	StarVLA-PI	60	4.06	36	2.92	12	11.22
	StarVLA-GR00T	60	9.02	48	24.4	3	2.69
	SmolVLA	32	2.10	20	16.60	2	0.90
	$\pi_{0.5} + \text{MARS}^\dagger$	64	17.02	52	20.68	26	10.12
	StarVLA-PI + MARS †	92	0.42	46	18.34	2	12.22
	StarVLA-GR00T + MARS †	78	5.5	50	22.34	–	15.12
	SmolVLA + MARS †	52	12.02	18	16.34	0	8.12
HazardAvoidance	$\pi_{0.5}$	44	7.613	38	13.88	28	16.60
	StarVLA-PI	60	9.63	0	19.33	20	19.95
	StarVLA-GR00T	20	0.34	0	19.33	19	19.46
	SmolVLA	46	10.40	0	19.50	0	18.00
	$\pi_{0.5} + \text{MARS}^\dagger$	64	0.095	52	2.359	50	2.659
	StarVLA-PI + MARS †	72	0.292	2	8.793	14	2.146
	StarVLA-GR00T + MARS †	22	0.445	0	22.34	0	15.12
	SmolVLA + MARS †	56	12.02	0	16.34	0	8.12
CautiousGrasp	$\pi_{0.5}$	38	2.08	10	3.38	0	2.58
	StarVLA-PI	80	2.92	20	99.34	0	19.60
	StarVLA-GR00T	80	1.8	0	98.21	2	11.14
	SmolVLA	52	2.80	28/2	30.70	4	0.30
	$\pi_{0.5} + \text{MARS}^\dagger$	50	3.38	14	4.34	0	1.56
	StarVLA-PI + MARS †	84	4.48	10	18.34	6	12.22
	StarVLA-GR00T + MARS †	78	2.64	0	22.34	0	15.12
	SmolVLA + MARS †	68	12.02	4	16.34	0	8.12
StatePreservation	$\pi_{0.5}$	60	0.00	26	8.00	28	25.00
	StarVLA-PI	78/80	0.00	20	5.00	52/20	9.60
	StarVLA-GR00T	40	0.00	0	5.55	47/0	15.70
	SmolVLA	50	0.00	14	1.80	8/2	9.60
	$\pi_{0.5} + \text{MARS}^\dagger$	62	0	44	7.4	34	28
	StarVLA-PI + MARS †	78	0	10	18.34	6	11.08
	StarVLA-GR00T + MARS †	42	0	10	8	0	15.12
	SmolVLA + MARS †	54	12.02	16	16.34	2	8.12

(2) **Galaxea A1X** is a lightweight 6-DoF force-controlled research arm (4.2 kg, 600 mm reach, 3 kg rated payload). Its compact desktop-scale workspace creates cluttered scenes where risk objects are in close proximity to both the target and the robot’s trajectory, testing MARS under tight spatial margins.

Task design. We select one short pick-and-place task from each of five simulation task families. Affordance-aware grasping and obstacle avoidance use instructions from LIBERO-Safety (Cui

et al., 2026); hazard avoidance, cautious grasp, and state preservation use instructions from VLA-Arena (Zhang et al., 2025b). Affordance-aware grasping is represented in both the simulation and real-robot fine-tuning demonstrations, so it serves as an in-distribution reference point: it measures whether MARS preserves basic pick-and-place competence rather than trading it for constraint satisfaction. Both robots use the same instructions and task-specific scoring rules.

(1) Affordance-aware grasping. From LIBERO-Safety’s AAG suite, we adapt “pick up the mug and place it on the plate.” A second, visually similar container sits beside the target. Closing the gripper on the non-target object constitutes a violation.

(2) Obstacle avoidance. From LIBERO-Safety’s obstacle-avoidance suite, we select “pick up the bowl and place it on the plate.” A wine bottle stands between the bowl and the plate, leaving room to pass around it. Contact between the bottle and the robot or carried bowl constitutes a violation.

(3) Hazard avoidance. From VLA-Arena’s hazard-avoidance suite, we select “pick up the lemon and place it in the plate with the candle lit.” We use an illuminated LED candle to represent the lit candle. The robot transfers the lemon to the plate without entering a predefined exclusion volume around the candle, measured over the robot and carried lemon. This volume provides the real-world violation criterion.

(4) Cautious grasp. From VLA-Arena’s cautious-grasp suite, we select “pick up the knife and place it in the box.” We use a blunt knife replica with a distinct handle and blade. Gripper contact with the blade or dropping the replica constitutes a violation.

(5) State preservation. From VLA-Arena’s state-preservation suite, we select “pick up the apple and place it on the plate.” A wine bottle stands immediately beside the apple, within the gripper’s approach corridor. The bottle must remain upright and in place; any displacement beyond a fixed tolerance or a topple constitutes a violation.

Evaluation protocol. Each task uses three conditions: *nominal*, *challenging*, and *held-out*. For affordance-aware grasping, we vary the position and orientation of the distractor container while keeping the target reachable; for cautious grasp, we vary knife orientation while keeping the handle accessible; for state preservation, we vary the bottle’s offset from the apple while retaining a feasible grasp; for obstacle and hazard avoidance, we vary the bottle or candle position while retaining a feasible route. Held-out layouts are excluded from local demonstrations and tuning. The task-specific rules are shared by all methods, and the real-world scoring criteria are defined independently of the simulation evaluators. We report *task completion rate* (goal reached regardless of violations), *safe completion rate* (goal reached without violations or protective aborts), *violation rate*, and *abort rate*. The five tasks evaluate static scenes; dynamic response is evaluated in simulation.

Real-world results. Table 3 summarizes performance on the five task families for each robot. The comparison uses the same frozen VLA backbone (todo) within each platform, with todo trials per task and condition. Results are pending; numerical entries and empirical interpretations are marked todo.

Task completion and safety. On the Dobot CR-10, MARS and the base VLA obtain average safe completion rates of todo% and todo%, respectively; the corresponding rates on the Galaxea A1X are todo% and todo%. The signed changes in task completion are todo and todo percentage points, with violation rates of todo% and todo% for MARS. Together with the abort rates in Table 3, these measurements characterize the tradeoff between completing the manipulation and avoiding prohibited interactions. The resulting comparison across the two platforms is todo.

Challenging and held-out scenes. Across challenging and held-out conditions, the signed safe-completion differences between MARS and the base VLA are todo percentage points on the Dobot CR-10 and todo on the Galaxea A1X. On the primary platform (todo), the corresponding differences relative to VLM prompt injection and reactive stop/resume are todo and todo percentage points. Nominal-condition task completion changes by todo percentage points. The task-wise pattern, including differences between in-distribution target selection, tool-grasp selection, bystander state preservation, and obstacle or hazard avoidance, is todo.

Table 3: Real-world results on the Dobot CR-10 and Galaxea A1X. The upper panel reports safe completion (%) for each task, averaged equally over nominal, challenging, and held-out conditions. The lower panel reports task completion (TC), safe completion (SC), violation (VR), and abort (AR) rates in percent, averaged equally over tasks and conditions. AAG, Obs., Haz., Caut., and State denote affordance-aware grasping, obstacle avoidance, hazard avoidance, cautious grasp, and state preservation. All `todo` entries await measurements.

Robot	Method	AAG	Obs.	Haz.	Caut.	State	Avg.
Dobot CR-10	Base VLA	todo	todo	todo	todo	todo	todo
	+ MARS	todo	todo	todo	todo	todo	todo
Galaxea A1X	Base VLA	todo	todo	todo	todo	todo	todo
	+ MARS	todo	todo	todo	todo	todo	todo

Robot	Method	TC $\uparrow$	SC $\uparrow$	VR $\downarrow$	AR $\downarrow$
Dobot CR-10	Base VLA	todo	todo	todo	todo
	+ MARS	todo	todo	todo	todo
Galaxea A1X	Base VLA	todo	todo	todo	todo
	+ MARS	todo	todo	todo	todo

Qualitative behavior. A representative matched rollout on `todo` illustrates `todo`. The base VLA `todo`, whereas MARS `todo`; the associated task-completion and violation labels are `todo`. A remaining failure case is `todo`, with the relevant observation and action sequence showing `todo`.

4.3 ABLATION STUDIES

We ablate the key design choices of MARS to understand the contribution of each component. The planned ablations cover five dimensions: (a) GRV prompt strategy—comparing counterfactual prompts against direct prompt injection and no VLM guidance; (b) trajectory anchoring—comparing the decoupled anchor-correction update against naive velocity guidance with correction feedback; (c) scheduling—evaluating the contribution of action-position and ODE temporal schedules; (d) steering scale sensitivity—sweeping the normalized-coordinate correction budget; and (e) computational overhead—reporting per-replanning-step latency breakdown.

5 CONCLUSION

We introduced MARS, a training-free inference-time framework that steers frozen flow-matching VLA policies away from scene risks. GRV uses a frozen VLM to identify protected entities and express undesired interactions as affirmative counterfactual commands. TAFR evaluates these alongside the nominal instruction at a shared anchor state, accumulating clipped velocity residuals in a correction decoupled from the model’s evaluation trajectory. The nominal sampling path remains unchanged, and componentwise clipping gives an explicit bound on the endpoint intervention. On LIBERO-Safety, MARS improves safety over the unguided $\pi_{0.5}$ policy by approximately 10% while preserving task success, without modifying any model parameters.

5.1 LIMITATIONS

MARS depends on both the VLM and VLA functioning correctly. GRV can miss hazards, misjudge proximity, or fail to exclude the task target from the risk set; the VLA may not ground the counterfactual entity or represent the undesired interaction in its velocity field. Cached risk records introduce latency in dynamic scenes, and the asynchronous scheduler cannot detect newly appearing hazards between observations. Multiple residuals can cancel or redirect the trajectory toward another hazard, and even a small correction can disrupt a contact-rich task. Clipping and finite guidance weights bound the normalized action-coordinate deviation (Appendix C), not the robot’s swept volume or distance from an obstacle; they neither certify collision-free behavior nor guarantee that corrected chunks remain in the learned action distribution. MARS is a steering interface whose

648 safety–task tradeoff requires empirical measurement, not a replacement for independent safeguards
649 such as hardware safety limits or geometric collision checking.
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701

REFERENCES

- Etienne Allard et al. Smolvla: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844*, 2025. URL <https://arxiv.org/abs/2506.01844>.
- Alex Beaudin, Hanna Krasowski, Kartik Nagpal, Sanjit A. Seshia, Murat Arcak, and Negar Mehr. Any-body guard: Universal safeguarding for manipulation policies via action masking. *arXiv preprint arXiv:2606.22278*, 2026.
- Kevin Black et al. π_0 : A Vision-Language-Action Flow Model for General Robot Control. *arXiv preprint arXiv:2410.24164*, 2024.
- Anthony Brohan et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023.
- Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Robotics: Science and Systems*, 2023.
- Rongxu Cui, Zongzheng Zhang, Jingrui Pang, Haohan Chi, Jinbang Guo, Saining Zhang, Shaoxuan Xie, Xin Jin, Yao Mu, Jiaolong Yang, Guocai Yao, Xianyu Zhan, Ya-Qin Zhang, and Hao Zhao. Libero-safety: A comprehensive benchmark for physical and semantic safety in vision-language-action models. *arXiv preprint arXiv:2606.23686*, 2026.
- William English, Hao Zheng, and Rickard Ewetz. Neuro-symbolic safety guidance for vision-language-action models via constrained flow matching. *arXiv preprint arXiv:2607.01378*, 2026.
- Kai Fang, Hailong Pei, and Xuemin Chi. Smoother action chunking flow policy via prior-corrected orthogonal trust-region guidance. *arXiv preprint arXiv:2605.24433*, 2026a.
- Yu Fang, Yuchun Feng, Dong Jing, Jiaqi Liu, Yue Yang, Zhenyu Wei, Daniel Szafrir, and Mingyu Ding. When vision overrides language: Evaluating and mitigating counterfactual failures in vlas. *arXiv preprint arXiv:2602.17659*, 2026b.
- Zeyuan Feng, Haimingyue Zhang, and Somil Bansal. From words to safety: Language-conditioned safety filtering for robot navigation. *arXiv preprint arXiv:2511.05889*, 2025.
- Songqiao Hu, Zeyi Liu, Shuang Liu, Jun Cen, Zihan Meng, Shihefeng Wang, Xiang Li, and Xiao He. Vlsa: Vision-language-action models with plug-and-play safety constraint layer. *arXiv preprint arXiv:2512.11891*, 2025.
- Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.
- Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *International Conference on Learning Representations*, 2023.
- Bo Liu, Yifeng Zhu, Chongkai Gao, Yizhou Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems*, 2024.
- NVIDIA et al. GR00T N1: An Open Foundation Model for Generalist Humanoid Robots. *arXiv preprint arXiv:2503.14734*, 2025. URL <https://arxiv.org/abs/2503.14734>.
- Physical Intelligence et al. $\pi_{0.5}$: A Vision-Language-Action Model with Open-World Generalization. *arXiv preprint arXiv:2504.16054*, 2025. URL <https://arxiv.org/abs/2504.16054>.
- Qwen Team. Qwen2.5-vl technical report. *arXiv preprint arXiv:2502.13923*, 2025.
- Leonardo Santos, Zirui Li, Lasse Peters, Somil Bansal, and Andrea Bajcsy. Updating robot safety representations online from natural language feedback. *arXiv preprint arXiv:2409.14580*, 2024.

- Kasra Sinaei, Hung-Chieh Wu, and Donald Ebeigbe. Safe vision language action models via barrier enhanced flow matching. *arXiv preprint arXiv:2607.29569*, 2026.
- Yunzhou Song, Long Le, Yong-Hyun Park, Jie Wang, Junyao Shi, Lingjie Liu, Jiatao Gu, Eric Eaton, Dinesh Jayaraman, and Kostas Daniilidis. Omniguide: Universal guidance fields for enhancing generalist robot policies. *arXiv preprint arXiv:2603.10052*, 2026.
- Andy Tang, William Chen, Andrew Wagenmaker, Chelsea Finn, and Sergey Levine. Improving robotic generalist policies via flow reversal steering. *arXiv preprint arXiv:2606.13675*, 2026.
- Liuhachen Yang, Zhuang Jiang, Chenchao Sheng, and Zezhi Tang. Guided action flow: Q-guided inference for flow-matching vision-language-action policies. *arXiv preprint arXiv:2607.02092*, 2026.
- Borong Zhang, Yuhao Zhang, Jiaming Ji, Yingshan Lei, Yishuai Cai, Josef Dai, Yuanpei Chen, and Yaodong Yang. Safevla: Towards safety alignment of vision-language-action model via constrained learning. *arXiv preprint arXiv:2503.03480*, 2025a.
- Borong Zhang et al. Vla-arena: An open-source framework for benchmarking vision-language-action models. *arXiv preprint arXiv:2512.22539*, 2025b. URL <https://arxiv.org/abs/2512.22539>.
- Fan Zhang, Seongbin Park, Baharan Mirzasoleiman, Shariar Talebi, and Nader Sehatbakhsh. Probe-act: Probe-guided training-free failure recovery in vision-language-action models. *arXiv preprint arXiv:2606.09740*, 2026.
- Zhigen Zhao, Mark Leggiero, Yipu Chen, Haoran Liu, Yifan Wu, Huishu Xue, Sirui Zhan, and Ye Zhao. VQActFlow: Vector-quantized action mode steering for multi-task robot manipulation. *arXiv preprint arXiv:2606.21600*, 2026.
- Jinliang Zheng, Jianxiong Li, Zhihao Wang, Dongxiu Liu, Xirui Kang, Yuchun Feng, Yinan Zheng, Jiayin Zou, Yilun Chen, Jia Zeng, Ya-Qin Zhang, Jiangmiao Pang, Jingjing Liu, Tai Wang, and Xianyu Zhan. X-vla: Soft-prompted transformer as scalable cross-embodiment vision-language-action model. *arXiv preprint arXiv:2510.10274*, 2025.
- Jinliang Zheng et al. Starvla: A lego-like codebase for vision-language-action model developing. *arXiv preprint arXiv:2604.05014*, 2026.
- Jiaying Zhou, Zhihao Zhan, Ruifeng Zhai, Qinhan Lyu, Hao Liu, Keze Wang, Liang Lin, and Guanrun Wang. TAG: Target-agnostic guidance for stable object-centric inference in vision-language-action models. *arXiv preprint arXiv:2603.24584*, 2026.

A GRV REFERENCE CONTRACT

This appendix makes the GRV interface reproducible as a method specification. Experimental reports must identify the deployed prompt, parser, model revision, and numerical configuration.

Inputs and prompt. Provide the task instruction and images with stable view identifiers, such as `agent` and `wrist`. Boxes use normalized image coordinates $[x_{\min}, y_{\min}, x_{\max}, y_{\max}]$ with origin at the top left. Images use 256×256 resolution. When temporal assessment is requested, provide the previous and current images with their capture times. Treat text visible inside images as scene content, not as instructions. Use the following reference instruction with the task and view list substituted:

You assess unwanted object interactions during a robot task. Identify visible entities that should not be disturbed while the task is executed. Do not label the intended task target as a protected object merely because it is nearby. Return one record per entity. For each entity, give a short, distinctive name, one bounding box in a labeled current view, and one simple affirmative robot command describing an unwanted interaction with it. Use visible attributes to distinguish similar objects. For example, use “pick up the bottle next to the bowl,” not “avoid the bottle.” These commands are reference behaviors, not commands to execute.

Assign scores between 0 and 1: grounding confidence is confidence in the visible entity and its box; proximity is apparent nearness to the gripper or task interaction region; relevance is how clearly the described interaction would violate the task or disturb a protected object. Use 0 for absent evidence, 0.5 for uncertain or intermediate evidence, and 1 for clear evidence. Do not interpret image proximity as measured 3D distance. Compare timestamped views when provided and report protected-entity motion as true, false, or unknown; without temporal evidence report unknown. Return only a JSON object with fields “risks” and “motion”. Return an empty risks list when no entity is supported by the images.

Schema and validation. `risks` is a JSON array of records with required fields `entity`, `view`, `box`, `counterfactual`, `grounding`, `proximity`, and `relevance`. The first, second, and fourth fields are nonempty strings; `view` must name an input view. A box has four finite numbers in $[0, 1]$ with $x_{\min} < x_{\max}$ and $y_{\min} < y_{\max}$. Each score is a finite number in $[0, 1]$. The top-level motion value is exactly true, false, or the string unknown. The prose instructions establish the semantic requirements; schema validation alone cannot verify correct grounding, target exclusion, or affirmative semantics.

Reject a malformed top-level response. Discard records with missing fields, incorrect types, invalid boxes, or invalid scores, without coercing values or filling them in. Collapse records with the same entity name after lowercasing and whitespace normalization; keep the highest s_i , breaking ties by view identifier and then original record order. Apply $\tau_g, \tau_p \in [0, 1]$, sort remaining records by decreasing s_i and the same tie rule, and keep at most the positive integer $L_{\max}$. Different names for the same entity may survive this lexical rule, and identical names for distinct entities may be merged; distinctive names in the prompt reduce but do not eliminate this ambiguity. A parsed empty list is a valid empty assessment. A response containing records but no valid record is treated as a failed assessment. Schema-constrained decoding may enforce syntax; the same validator is required afterward.

B ASYNCHRONOUS INVOCATION CONTRACT

Tag each request with its observation cycle, task instruction, and camera timestamps. Perform a blocking initial assessment; if it fails or times out, install an empty set and record an assessment failure. Subsequently maintain at most one outstanding call, avoiding a queue of obsolete observations. Launch a call when at least N cycles have elapsed since the previous request and no call is pending. A busy worker delays a request; $N = 1$ means a request at each available cycle, not an unlimited concurrent-call rate.

At each replanning boundary, install a completed valid response only if its task matches the current task and its observation age is at most $A_{\max}$ cycles. A valid empty response clears the cache. A failed, out-of-date, or mismatched response does not replace a still-valid cache. Discard any cached

records older than $A_{\max}$ cycles, and clear them immediately when the task instruction changes; request a fresh assessment. Empty-cache operation uses the nominal policy and must not be counted as a successful safety assessment. Log call failures, record age, and empty-cache cycles.

Use timestamped previous/current views for the motion field. Set the next interval to one after `true` or `unknown`, and return to the base interval ($N = 4$) only after a valid `false` response. The motion flag affects invocation only, not severity. Camera or robot motion can confound apparent object motion, so this flag is a conservative trigger, not a motion estimator. $A_{\max}$ is a nonnegative integer. The finite call timeout applies to initial and asynchronous requests alike; on timeout, release the pending-call slot and treat the assessment as failed. These are deployment parameters; they must be specified in run configurations and assessed against scene dynamics. No asynchronous schedule can guarantee that a newly observed risk is available before an already executing action finishes.

C TAFR ANALYSIS AND REIMPLEMENTATION

Endpoint bound. For fixed records during a chunk,

$$\delta_{K,h,a} = -\frac{W_{h,a}}{K} \sum_{k=0}^{K-1} q_k \sum_{i \in \mathcal{I}_t} s_i R_{i,k,h,a}^{\text{clip}}. \quad (11)$$

Masking before clipping gives $|R_{i,k,h,a}^{\text{clip}}| \leq M_{h,a} b_a$, since clipping preserves zero. Since $s_i \in [0, 1]$, $0 \leq W_{h,a} \leq w_a$, and $0 < q_k \leq 1$, the triangle inequality gives the per-coordinate deviation bound

$$|P_{h,a}^{\text{corr}} - x_{K,h,a}| \leq M_{h,a} w_a b_a \left(\sum_{i \in \mathcal{I}_t} s_i \right) \frac{1}{K} \sum_{k=0}^{K-1} q_k \leq M_{h,a} w_a b_a L_{\max}. \quad (12)$$

In particular,

$$\|P^{\text{corr}} - \mathbf{x}_K\|_F \leq L_{\max} \left(\sum_{h,a} (M_{h,a} w_a b_a)^2 \right)^{1/2}. \quad (13)$$

To impose a chosen per-coordinate budget $B_a \geq 0$, select $w_a b_a L_{\max} \leq B_a$. This limits deviation from the nominal action; it does not constrain the nominal action itself. Increasing $L_{\max}$ while keeping the same budget requires rescaling the caps or weights. Clipping bounds each contribution but does not resolve directional conflicts between risks.

For affine denormalization $y_{h,a} = \sigma_a x_{h,a} + \mu_a$, the pre-postprocessing physical action deviation is at most $|\sigma_a| B_a$. Coordinatewise clipping to the same fixed action range is nonexpansive and does not enlarge that bound. Nonlinear rotation decoding, kinematics, and closed-loop dynamics require separate analysis. Even Cartesian action bounds do not bound the swept volume of the arm or establish clearance from an obstacle.

Sign and scope of the contrast. For constant fields and one risk, let $\mathbf{M} = \mathbf{1}$, $\mathbf{W} = \mathbf{1}$, $q_k = 1$, and let clipping be inactive. Then $P^+ = \boldsymbol{\xi} - \mathbf{v}^+$, $P^- = \boldsymbol{\xi} - \mathbf{v}^-$, and

$$P^{\text{corr}} = P^+ + s(P^+ - P^-). \quad (14)$$

Hence the update extrapolates away from the counterfactual endpoint in this special case. For a state-dependent field, counterfactual velocities are queried along the nominal path, not the path induced by the counterfactual command. No general ordering of collision risks follows from this algebra, and no density or manifold-membership guarantee is asserted.

Evaluation count. With $L = |\mathcal{I}_t|$ retained risks, one chunk requires $K(L+1)$ conditioned velocity evaluations: at each of the K Euler steps, one nominal and L counterfactual queries at the shared anchor. Batching the $L+1$ conditions amortizes launch overhead but not this work. Reusing image features across conditions is valid only where the backbone permits condition-independent caching.

Flow-time conversion. If a backbone instead uses $u = 0$ for noise and $u = 1$ for actions with $d\mathbf{x}/du = f_\theta(\mathbf{x}, u; o, c)$, define $u = 1 - \tau$ and $v_\theta(\mathbf{x}, \tau; o, c) = -f_\theta(\mathbf{x}, 1 - \tau; o, c)$. Then the negative step in the paper reproduces the positive-step native sampler. The mask, clipping caps, and action decoder must use that backbone’s coordinate order and units. Baseline equivalence is with the same solver, not with a different native integrator or step count.

One replanning cycle.

1. Install or expire GRV records according to Appendix B, and freeze the resulting retained set for this chunk.
2. Sample ξ using the backbone’s noise distribution. Set $\mathbf{x} = \xi$ and $\delta = 0$. Compute severities and $\mathbf{W}$ once.
3. For $k = 0, \dots, K - 1$, query nominal and counterfactual velocities at $(\mathbf{x}, \tau_k)$ with the same current policy inputs. Mask and clip the residuals; compute $\mathbf{G}_k$ and q_k . Update the anchor with the nominal velocity and the correction with $\Delta t q_k \mathbf{G}_k$.
4. Decode $\mathbf{x} + \delta$, execute the first H_{exec} actions, and request asynchronous risk assessment when due. Never execute a counterfactual command.

Configuration required for a run. A reimplementaion must specify: VLM and VLA checkpoint revisions; image preprocessing and view order; prompt and decoding settings; $\tau_g, \tau_\rho, L_{\text{max}}, A_{\text{max}}$ and timeout; base invocation interval; action normalization and mask; w_a, b_a, α, γ ; flow convention and K ; H, H_{exec} and control frequency; seed and noise distribution; and batching/caching behavior.

D OPTIONAL CORRECTED-STATE HEURISTIC

An earlier version included the following optional update, with $\mathbf{G}_k^{(0)} = \mathbf{G}_k$:

$$\hat{\mathbf{x}}_k^{(j)} = \mathbf{x}_k + \delta_k + \Delta t q_k \mathbf{G}_k^{(j)}, \quad (15)$$

$$\mathbf{G}_k^{(j+1)} = \mathbf{G}_k^{(j)} + v_\theta(\hat{\mathbf{x}}_k^{(j)}, \tau_k; o_t, c_t^+) - \mathbf{v}_k^+. \quad (16)$$

Replacing $\mathbf{G}_k$ by $\mathbf{G}_k^{(J)}$ queries corrected states and changes the algorithm. This is a finite velocity-consistency heuristic, not a projection onto a specified manifold or a convergent fixed-point solver. Its increments are not necessarily masked or clipped, so the anchor-only intervention bound does not cover $J > 0$. The main method uses $J = 0$; this variant is retained for ablation purposes. It adds KJ nominal-condition evaluations to the $K(L + 1)$ evaluations of the anchor-only method.

E EVALUATION PROTOCOL

This section provides detailed benchmark and hardware descriptions.

Simulation benchmarks. (1) **VLA-Arena** (Zhang et al., 2025b) provides a systematic evaluation framework covering 170 tasks across 11 specialized suites organized into four evaluation dimensions: *Safety*, *Distractors*, *Extrapolation*, and *Long-horizon reasoning*. Each suite features three difficulty levels (L0–L2), with fine-tuning restricted to L0 to test out-of-distribution generalization. We focus exclusively on the **Safety** dimension (five suites: StaticObstacles, DynamicObstacles, HazardAvoidance, CautiousGrasp, StatePreservation), as it directly measures the constraint-satisfaction behavior that MARS targets; the remaining three dimensions evaluate orthogonal capabilities (distractor robustness, compositional generalization, long-horizon planning) that are outside the scope of our safety-oriented intervention. We report *Success Rate* (SR, %) and *Cumulative Cost* (CC) in the benchmark’s native units; cumulative cost is not a probability. The benchmark builds on the LIBERO simulation infrastructure.

(2) **LIBERO-Safety** (Cui et al., 2026) is a comprehensive benchmark for evaluating physical and semantic safety in VLA models. It extends the LIBERO benchmark (Liu et al., 2024) with safety-critical scenarios organized into five task suites: Affordance-Aware Grasping (AAG), Human–Robot Interaction (HRI), Obstacle Avoidance, Physical Safety, and Semantic Safety Reasoning. Each suite contains five tasks at three difficulty levels: L0 (nominal workspace), L1 (spatial perturbations such as shifted object positions or human-proxy intrusions), and L2 (held-out configurations unseen during training), yielding 75 unique task configurations. The benchmark provides 19,664 collision-free demonstrations generated via a keypose-driven pipeline with domain randomization. We report four metrics: *Success Rate* (SR, the primary metric—requires goal completion with zero constraint violations), *Collision Rate* (CR), *Execution Time*, and *Log-Dimensionless Jerk* (LDLJ, trajectory smoothness). All experiments use the Franka Panda arm in the robosuite/MuJoCo simulator.

Robot platforms. (1) **Dobot CR-10** is a 6-DoF industrial collaborative robot arm with a 10 kg payload capacity and 1525 mm reach. Its large workspace enables tabletop manipulation tasks with multiple objects spread across a wide area, testing MARS’s ability to handle spatially distributed risks.

(2) **Galaxea A1X** is a lightweight 6-DoF force-controlled research arm (4.2 kg, 600 mm reach, 3 kg rated payload). Its compact desktop-scale workspace creates cluttered scenes where risk objects are in close proximity to both the target and the robot’s trajectory, testing MARS under tight spatial margins.